

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a079fabb51dd26ad7.jsonl`  
- SHA-256: `72be80fea8cfb0ecb3a18100af85e1b6705983a5bc8d35c336655664b7dc53ea`  
- Source modified: `2026-07-02T04:58:34+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_7049e1ed-b93`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:53:47.798Z)

You are an adversarial code reviewer. Today is 2026-07-02. Review PR #415 in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (gh pr view 415, gh pr diff 415; the working copy is in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-upload-dir ? read code there for context).

The PR claims to fix: R1-19 (upload_dir CWD-relative) (evidence in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep for the IDs; honor the verifier notes' scoping corrections). Hunt REAL defects: correctness bugs in the diff, broken/vacuous tests, behavior changes beyond the brief, staging accidents (files that should not be in the diff ? check the file list!), convention violations (default-ON when brief says OFF, missing schema-version check, CWD assumptions), and whether the fix actually addresses the finding. READ-ONLY ? do not modify or comment on GitHub. blocker = must not merge; major = fix before merge; minor = nice-to-have.

2. user (2026-07-02T04:53:54.892Z)

```
title: api: root a relative security.upload_dir under the app home (P0a parity with output_dir)  
state: OPEN  
author: avidullu (Avi)  
labels:  
assignees:  
reviewers:  
projects:  
milestone:  
number: 415  
url: https://github.com/Khelsutra/badminton-highlight-indexer/pull/415  
additions: 80  
deletions: 5  
auto-merge: disabled  
--  
<!-- Audit 2026-07-02, finding R1-20 (upload_dir CWD-relative; the task brief labeled it R1-19).  
-->
```

Summary

`security.upload\_dir` defaulted to the CWD-relative `output/uploads` and was **never** passed through the P0a app-home resolver ? only `output.output\_dir` was resolved (`backend/main.py:1992`). So the packaged `indexer --app` (which `scripts/install.py` advertises as launchable from any shell) wrote browser uploads into whatever CWD the shell happened to have ? or 500'd on an unwritable one ? while the SQLite DB and reels correctly lived under `RALLY\_APP\_HOME`. Exactly the CWD-vs-app-home class of bug P0a was built to eliminate.

Fix: `backend/api/uploads.py::upload\_cfg` now resolves `upload\_dir` through the same

`resolve\_output\_dir` (`backend/utils/app\_home.py`) used for `output.output\_dir`, mirroring its semantics exactly:

- `RALLY\_APP\_HOME` **unset** ? the configured value verbatim (CWD-relative, byte-identical to before ? dev/repo/CI paths unchanged);
- **set** ? a relative `upload\_dir` (incl. the default) roots under the home (`<home>/output/uploads`);
- an **absolute** (or Windows drive-anchored) `upload\_dir` always passes through untouched.

Resolving in `\_upload\_cfg` (rather than once in `main()`) keeps every entry point covered ? including apps built via `create\_app()` without going through `main()` ? and `app\_home()` is stable per-process, so the result is deterministic. Docs: `SecurityConfig.upload\_dir` comment + `docs/CODE\_MAP.md` §2.0 updated to match.

Tests (`tests/test\_upload\_download.py`, new P0a section):

- relative default resolves under the app home when `RALLY\_APP\_HOME` is set;
- absolute `upload\_dir` passes through even with the env set;
- env unset ? verbatim `output/uploads` (pins the unchanged pre-P0a default);
- end-to-end: init?part?complete from an arbitrary CWD lands the assembled file under `/output/uploads` and leaks nothing into the launch CWD.

No existing test pinned the CWD-relative behavior of the *default* (the fixture uses an absolute `upload\_dir`, which passes through unchanged), so no test needed loosening.

Verification

Ran locally (Windows, worktree off `origin/master` @ e410136):

- `python -m pytest tests/test\_upload\_download.py tests/test\_app\_home.py -q` ? **46 passed**
- `ruff check` + `ruff format --check` on the changed files ? clean
- `python -m mypy backend/api/uploads.py backend/config/models.py` ? no issues
- `python -m tools.check\_md\_links` ? clean

CI covers the full `run\_all\_tests.py` suite + coverage floor + link-check. Not verified locally: a real packaged `indexer --app` launch (install-flow/GPU machine checks stay with the owner per CLAUDE.md); the end-to-end test simulates that shape via `RALLY\_APP\_HOME` + `monkeypatch.chdir`.

Checks

- [x] Targeted tests green; `ruff check` + `mypy` clean on changed files; full suite + coverage in CI.
- [x] Internal markdown links valid (`python -m tools.check\_md\_links`).

? Generated with [Claude Code](https://claude.com/claude-code)

3. user (2026-07-02T04:53:56.140Z)

diff --git a/backend/api/uploads.py b/backend/api/uploads.py

index ccb1511..30be0b1 100644

--- a/backend/api/uploads.py

+++ b/backend/api/uploads.py

@@ -3,6 +3,9 @@

Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side under security.upload_dir, then submitted to /api/process by the returned path.

+A RELATIVE upload_dir roots under the app home (RALLY_APP_HOME) via the same

+`resolve\_output\_dir` used for output.output_dir ? P0a semantics: env unset ?

+verbatim (CWD-relative, byte-identical to before); absolute ? passes through.

Upload state is in-process by design (this is the single-box alpha): a server restart aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir arrives with PR-3 identity scoping.

@@ -21,6 +24,7 @@

from fastapi import APIRouter, HTTPException, Request

from pydantic import BaseModel

+from backend.utils.app_home import resolve_output_dir # P0a app-home rooting

from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)

from backend.utils.hashing import compute_video_id # canonical file-identity hashing

```

from backend.utils.paths import safe_output_filename
@@ -47,7 +51,14 @@ def _upload_cfg():
    import backend.main as _main

    sec = getattr(_main.global_config, "security", None)
-   upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
+   # P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like
+   # main() resolves output.output_dir ? the packaged `indexer --app` launches from an
+   # arbitrary shell CWD, and the upload landing zone must live with the DB/reels, not
+   # wherever the shell happened to be. Env unset ? verbatim (CWD-relative, unchanged);
+   # an absolute upload_dir always passes through untouched.
+   upload_dir = resolve_output_dir(
+       getattr(sec, "upload_dir", None) or "output/uploads"
+   )
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [
diff --git a/backend/config/models.py b/backend/config/models.py
index 652693a..e6b13e3 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -330,9 +330,12 @@ class SecurityConfig(BaseModel):
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
-   # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
-   # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
-   # contain upload_dir or /api/process will reject the uploaded paths.
+   # Landing zone for browser uploads (assembled from parts). A RELATIVE path roots
+   # under the app home (RALLY_APP_HOME) at use time ? same P0a resolve_output_dir
+   # semantics as output.output_dir (env unset ? CWD-relative verbatim; absolute ?
+   # passes through). Per-user subdirectories arrive with PR-3 identity scoping.
+   # ? In hosted mode, set allowed_video_root to contain upload_dir or /api/process
+   # will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
diff --git a/docs/CODE_MAP.md b/docs/CODE_MAP.md
index 1aa5260..2657218 100644
--- a/docs/CODE_MAP.md
+++ b/docs/CODE_MAP.md
@@ -112,7 +112,7 @@ Entrypoints: `@run_eval.py` (single video score),
`@scripts/run_phase3_eval.py`
-   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI
mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
-   - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module globals
set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every
endpoint NPEs.
-   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+   job_id**. `::drain_due_jobs` / `::run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`

```

```

`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the
pipeline synchronously inline (separate code path, unchanged by #16).
- - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
+ - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB). A RELATIVE `upload_dir` is rooted under the app home via
`resolve_output_dir` in `upload_cfg` (P0a ? same semantics as `output.output_dir`:
`RALLY_APP_HOME` unset ? CWD-relative verbatim; absolute ? passthrough), so packaged `indexer
--app` uploads land with the DB/reels regardless of the launch CWD.
- **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET
/api/capabilities` ? honest box probe; now includes a `**storage` block**
(`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive
`{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment.
`::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET
/api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy**
(D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes
the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset`
`@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a
video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-
in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in
`/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-
auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`,
`::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):**
`@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +
the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::wsl_frame_stage_shortfall`, ~15x source).
- `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly.
`::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest`
pydantic bodies.
- `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.

```

```
compile_highlights` reads the SAME `global_config.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).  
diff --git a/tests/test_upload_download.py b/tests/test_upload_download.py  
index 6be9145..9566e6b 100644  
--- a/tests/test_upload_download.py  
+++ b/tests/test_upload_download.py  
@@ -9,6 +9,7 @@  
 """"  
  
 import os  
+from pathlib import Path  
 from unittest.mock import patch  
  
 import pytest  
@@ -20,6 +21,7 @@  
 from backend.api import uploads as uploads_api  
 from backend.config.models import AppConfig  
 from backend.infrastructure.database import Database  
+from backend.utils import app_home as ah  
 from backend.utils.hashing import compute_video_id  
  
@@ -233,6 +235,65 @@ def test_abort_endpoint_removes_state(env):  
     ]  
  
+# --- P0a: upload_dir roots under the app home (RALLY_APP_HOME) -----  
+# Same resolve_output_dir semantics as output.output_dir: env unset ? verbatim  
+# (CWD-relative, pre-P0a behaviour); env set ? relative roots under the home,  
+# absolute passes through. Packaged `indexer --app` launches from any shell CWD.  
+  
+def test_upload_cfg_relative_default_roots_under_app_home(monkeypatch, tmp_path):  
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))  
+    monkeypatch.setattr(m, "global_config", AppConfig()) # upload_dir at its default  
+    upload_dir, _, _, _ = uploads_api.upload_cfg()  
+    assert upload_dir == str(tmp_path / "output" / "uploads")  
+  
+def test_upload_cfg_absolute_upload_dir_passes_through(monkeypatch, tmp_path):  
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))  
+    cfg = AppConfig()  
+    abs_dir = str(tmp_path / "elsewhere")  
+    cfg.security.upload_dir = abs_dir  
+    monkeypatch.setattr(m, "global_config", cfg)  
+    upload_dir, _, _, _ = uploads_api.upload_cfg()  
+    assert upload_dir == abs_dir  
+  
+def test_upload_cfg_env_unset_keeps_cwd_relative_default(monkeypatch):  
+    # RALLY_APP_HOME unset ? byte-identical to the pre-P0a CWD-relative default.  
+    monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)  
+    monkeypatch.setattr(m, "global_config", AppConfig())  
+    upload_dir, _, _, _ = uploads_api.upload_cfg()  
+    assert upload_dir == "output/uploads"  
+  
+def test_upload_lands_under_app_home_not_launch_cwd(tmp_path, monkeypatch):  
+    # End-to-end packaged shape: RALLY_APP_HOME set, upload_dir left at its relative  
+    # default, server "launched" from an arbitrary shell CWD ? the assembled upload  
+    # must land under <home>/output/uploads and leak nothing into the CWD.  
+    home = tmp_path / "home"  
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(home))  
+    cwd = tmp_path / "some-shell-cwd"  
+    cwd.mkdir()
```

```

+ monkeypatch.chdir(cwd)
+ db = Database(str(tmp_path / "t.db"))
+ cfg = AppConfig() # security.upload_dir stays 'output/uploads'
+ cfg.output.output_dir = str(tmp_path / "out")
+ os.makedirs(cfg.output.output_dir, exist_ok=True)
+ monkeypatch.setattr(uploads_api, "_uploads", {})
+ client = TestClient(m.create_app(cfg, db))
+ up = client.post("/api/upload/init", json={"filename": "match.mp4"}).json()
+ r = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"match-bytes")
+ assert r.status_code == 200
+ done = client.post(
+     f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 1}
+ )
+ assert done.status_code == 200
+ final = Path(done.json()["path"])
+ assert final.parent.resolve() == (home / "output" / "uploads").resolve()
+ assert final.read_bytes() == b"match-bytes"
+ assert not (cwd / "output").exists() # nothing written into the launch CWD
+
+ # --- download (B3) -----

```

4. user (2026-07-02T04:53:56.998Z)

```

204-
205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206-**GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute());
207-put() deletes the old object before the new upload succeeds**
208-- anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder: engine:correctness
209-- evidence: gdrive_api.py:143-148: `data =
210-  self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
211-object is materialized as one bytes value (no MediaIoBaseDownload streaming).
212-gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
213-BEFORE `files().create(...)` ? delete-then-create is not atomic.
214-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes
215-live for the reel round-trip.
216-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks;
217-for put(), create the new file first (or upload to a temp name) and delete the old one only
218-after success.
219-[Omitted long context line]
220-[Omitted long context line]
221-
222-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
223-**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on
224-every server/CLI start ? the typed knob is dead, and the DB path moves with it**
225-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
226-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992
227-unconditionally does `global_config.output.output_dir = resolve_output_dir(args.output_dir)` ?
228-the override fires even when the flag was never passed, so a config.json value can never take
229-effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir
230-overrides this at startup' ? implying only when given). db_path derives from it
231-(main.py:1995-1997).
232-- why it matters: A documented config knob that is a no-op on the main entry path, with
233-data-location consequences (perceived data loss / split state).
234-- proposed fix: Change argparse default to None and only override when args.output_dir is
235-not None (falling back to the config value through resolve_output_dir).
236-[Omitted long context line]
237-[Omitted long context line]
238-
239-### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
240-**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged
241--app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live
242-under RALLY_APP_HOME**

```

5. user (2026-07-02T04:54:04.639Z)

```

223:### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
224-**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged
--app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live
under RALLY_APP_HOME**
225-- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness
226-- evidence: uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or
'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no
resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved,
main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.
227-- why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly
the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).
228-- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg
(relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229-[Omitted long context line]
230-[Omitted long context line]
231-
232:### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer
233-**Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no
cancellation, so the L4 keeps billing and its late result is never ingested**
234-- anchor: `backend/compute/cloud_run.py:173` · tracked in: ? · finder: engine:correctness
235-- evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...);
execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The
exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ?
no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the
late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).
236-- why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-
gated go-live area, but the code path is live behind compute='cloud').
237-- proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API
(executions.cancel with the stored execution name) and log the execution id in the failure
message; optionally check the done-marker once more before failing.
238-[Omitted long context line]

```

6. user (2026-07-02T04:54:07.205Z)

```

backend/api/uploads.py
backend/config/models.py
docs/CODE_MAP.md
tests/test_upload_download.py

```

7. user (2026-07-02T04:54:16.271Z)

```

205    ### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206    **GDriveApiStorage.fetch_to_local buffers the entire file in RAM
(get_media().execute()); put() deletes the old object before the new upload succeeds**
207    - anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder:
engine:correctness
208    - evidence: gdrive_api.py:143-148: `data =
self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
BEFORE `files().create(...)` ? delete-then-create is not atomic.
209    - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it
goes live for the reel round-trip.
210    - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in
chunks; for put(), create the new file first (or upload to a temp name) and delete the old one
only after success.
211    - verifier notes: File/line accurate (line 144 is the get_media().execute() call).
Failure scenarios: (1) fetch_to_local on a multi-GB source/reel buffers the full file in RAM on
a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes
the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at
either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media_body=...)`
when a file already exists ? atomic in-place content replacement that preserves the file id and
avoids both the delete-first data-loss window and the duplicate-(name,parent) window that
create-first-then-delete would open (the file's own comment at lines 160-162 notes _resolve_id
returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked

```

streaming.

212 - verifier reasoning: backend/storage/gdrive_api.py:143-148 ? fetch_to_local does `data = self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)`: the whole object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine OOM risk. backend/storage/gdrive_api.py:163-170 ? put() resolves `existing`, calls `files().delete(fileId=existing).execute()`, and only then `files().create(...)` : a create failure after the delete loses the prior object (non-atomic overwrite). git log -- backend/storage/gdrive_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate (lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.

213

214 ### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer

215 **config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**

216 - anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness

217 - evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).

218 - why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).

219 - proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).

220 - verifier notes: Path correction: the documented knob is at backend/config/models.py:274-276, not backend/models.py (which does not exist). Sharper failure scenario: user sets output.output_dir: "D:/rally-data" in config.json and restarts `indexer --app`; main.py:1992 resets the dir to resolve_output_dir("output"), a fresh empty DB is created there (main.py:1995-1997) so the UI shows zero videos (perceived data loss), while run_eval.py/run_regression.py via backend/config/__init__.py:41-42 resolve the DB at D:/rally-data ? server and eval tooling disagree on DB location. Fix scope addition: the trim path at backend/main.py:1927-1929 also calls resolve_output_dir(args.output_dir) before config load and must handle a None default (fall back to the loaded config value there too, or to "output" since global_config isn't initialized yet at that point). Effort stays S.

221 - verifier reasoning: Read current tree. backend/main.py:1837-1841 defines --output-dir with default="output". backend/main.py:1982 loads the typed config; backend/main.py:1992 then unconditionally does global_config.output.output_dir = resolve_output_dir(args.output_dir), which fires even when the flag was never passed. resolve_output_dir (backend/utis/app_home.py:83-100) never consults the config value ? with RALLY_APP_HOME unset it returns the argparse default "output" verbatim ? so a config.json output.output_dir can never take effect. db_path derives from the clobbered value (backend/main.py:1995-1997). The only production entry is main() ? create_app (main.py:2184) ? uvicorn.run (main.py:2185); there is no module-level app, so every server/CLI start hits the clobber. The knob is documented at backend/config/models.py:274-276. Not fixed: git log --oneline -15 -- backend/main.py tops out at e410136 (#413), none touching this. Split-state is concrete: backend/config/__init__.py:41-42 (default_db_path, used by run_eval.py/run_regression.py) builds the DB path directly from config.json's output.output_dir with no clobber ? so with the knob set, the server and the eval tooling resolve DIFFERENT DB locations. Not owner-gated (local config/CLI startup only).

222

223 ### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer

224 **upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live under RALLY_APP_HOME**

225 - anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness

226 - evidence: uploads.py:50 `upload\_dir = getattr(sec, 'upload\_dir', None) or 'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved, main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.

227 - why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).

228 - proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg (relative ? app-home-rooted; absolute passes through), mirroring output_dir.

229 - verifier notes: Sharper failure scenario: on macOS the .command runs with Terminal's CWD=\$HOME, and any `indexer --app` shell launch (advertised in install.py:243) uses the caller's CWD ? uploads land in e.g. ~/output/uploads while DB/reels live under ~/Library/Application Support/Khelsutra/output; unwritable CWD ? PermissionError at uploads.py:89 ? 500 on /api/upload/init. Caveat: Windows double-click of KhelSutra.bat coincidentally works (the .bat lives in app_home, install.py:175, and cmd sets CWD to the .bat dir). Extra impact the finder missed: tools/cleanup_caches.py:403 sweeps <output-dir>/uploads (app-home-rooted post-P0a) and the uninstall manifest user_data (install.py:117-121) lists only app-home paths, so stray multi-GB uploads are never swept and survive uninstall --purge. Fix as proposed is sane: resolve_output_dir() in _upload_cfg is byte-identical with RALLY_APP_HOME unset (app_home.py:94-95) and existing tests use absolute tmp paths (test_upload_download.py:31) that pass through.

230 - verifier reasoning: uploads.py:50 `upload\_dir = getattr(sec, "upload\_dir", None) or "output/uploads"` used verbatim at :89 os.makedirs, :93, :95, :160, :170 ? no resolver applied (repo-wide grep). main.py:1992 resolves ONLY output.output_dir via resolve_output_dir (app_home.py:83-100); models.py:336 default is relative "output/uploads"; the packaged launcher (install.py:81-99) sets RALLY_APP_HOME but never cd's, and the first-run seeded config (loader.py:261-265 LIGHT_DEFAULT_CONFIG) has no security block, so the relative default is live in every packaged install. git log -- backend/api/uploads.py (top f28af73) shows no fix. Not owner-gated (local packaging correctness).

231

232 ### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer

233 **Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no cancellation, so the L4 keeps billing and its late result is never ingested**

234 - anchor: `backend/compute/cloud\_run.py:173` · tracked in: ? · finder: engine:correctness

235 - evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...); execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ? no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).

236 - why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-gated go-live area, but the code path is live behind compute='cloud').

237 - proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API (executions.cancel with the stored execution name) and log the execution id in the failure message; optionally check the done-marker once more before failing.

238 - verifier notes: Correction to the failure scenario: "the L4 keeps billing" is bounded, not open-ended ? the job is created with --max-retries 0 --task-timeout 3600 (deploy/cloudrun/README.md:33), so the orphaned execution bills at most ~30 extra minutes past the 1800s poll deadline, then Cloud Run kills it. The sharper, systematic scenario is the policy/task-timeout mismatch (1800s poll vs 3600s task budget): a long match video whose GPU inference takes 30-60 min ALWAYS ends as a local "cloud dispatch error" failure while the cloud run later succeeds, writes the done-marker + outputs to the bucket, and the result is silently wasted ? user pays full GPU cost and must re-dispatch. Fix scoping corrections: (a) cancellation must be a new abstract method on CloudRunJobClient (the injected test seam) with the real impl using run_v2.ExecutionsClient().cancel_execution, owner-verified like run_job since the SDK is not a CI dep; (b) it must be best-effort ? run_job's return falls back to the literal string "execution" when op.metadata.name is absent (cloud_run.py:108-112), which is not a resolvable execution name; (c) the "check the marker once more before failing" suggestion is good and cheap; also consider aligning max_wait_sec with the job's task-timeout. Owner-gating: the shim fix + fake-client tests are doable now; only the real-client cancel verification and any policy retune on the live job are owner-gated (M7 real-run / real cloud spend convention), which matches the finding's own caveat.

239 - verifier reasoning: Personally reproduced from current code (post-pull, HEAD e410136).

(1) backend/compute/cloud_run.py:173-178 ? run_infer blocks on poll_until(self._read_marker(done_uri), self._policy) with no exception handling; backend/infrastructure/execution_policy.py:54 sets max_wait_sec=1800.0 default and :183-189 raises PolicyTimeout at the deadline. (2) backend/main.py:1064-1070 catches the exception and returns _failed(f"cloud dispatch error: {e}", True) ? the local job goes terminal-failed, so the late done-marker/outputs are never ingested. (3) No cancellation exists anywhere: grep for cancel/executions./CancelExecution across backend/*.py hits only the minified UI asset (backend/ui/assets/index-ukuS9wxa.js); GoogleCloudRunJobClient (cloud_run.py:69-112) only implements run_job, and the CloudRunJobClient ABC (:50-66) has no cancel method. (4) Not already fixed: git log -15 -- backend/compute/cloud_run.py shows only formatting (b6214e5), isort (f523d3a), an unrelated project-guard fix (688d28f), and the original M6/provenance commits. (5)

The mechanism is worse than a rare edge: `get_compute_backend` (`backend/compute/__init__.py:39,76`) passes `DEFAULT_POLICY` (1800s) while the deployed job uses `--task-timeout 3600` (`deploy/cloudrun/README.md:33`, `docs/CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:57`), so any execution legitimately taking 30-60 min is systematically abandoned at 30 min, keeps billing the L4 for up to another 30 min, then its completed result is dropped. `deploy/cloudrun/README.md:60-64` documents the timeout path but only for hung/crashed containers ? the slow-but-healthy case was not considered.

8. user (2026-07-02T04:54:24.722Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      A RELATIVE upload_dir roots under the app home (RALLY_APP_HOME) via the same
7      ``resolve_output_dir`` used for output.output_dir ? P0a semantics: env unset ?
8      verbatim (CWD-relative, byte-identical to before); absolute ? passes through.
9      Upload state is in-process by design (this is the single-box alpha): a server restart
10     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
11     arrives with PR-3 identity scoping.
12
13     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
14     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
15     reads the live value the server set at startup (and so tests that monkeypatch
16     ``backend.main.global_config`` flow through unchanged).
17     """
18
19     import os
20     import threading
21     import uuid
22     from typing import Any, Dict, Optional
23
24     from fastapi import APIRouter, HTTPException, Request
25     from pydantic import BaseModel
26
27     from backend.utils.app_home import resolve_output_dir # P0a app-home rooting
28     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.utils.paths import safe_output_filename
31
32     router = APIRouter()
33
34     _uploads: Dict[str, Dict[str, Any]] = {}
35     _uploads_lock = threading.Lock()
36
37
38     class UploadInitRequest(BaseModel):
39         filename: str
40         total_size_bytes: Optional[int] = None
41
42
43     class UploadCompleteRequest(BaseModel):
44         total_parts: int
45
46
47     def _upload_cfg():
48         # Resolved lazily (import inside the function) to read the LIVE startup config that
49         # main() sets on the module global, and to avoid a circular import at module load
50         # (main.py imports this router; this module must not import main at import time).
51         import backend.main as _main
52
53         sec = getattr(_main.global_config, "security", None)
54         # P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like
55         # main() resolves output.output_dir ? the packaged `indexer --app` launches from an
```

```

56     # arbitrary shell CWD, and the upload landing zone must live with the DB/reels, not
57     # wherever the shell happened to be. Env unset ? verbatim (CWD-relative, unchanged);
58     # an absolute upload_dir always passes through untouched.
59     upload_dir = resolve_output_dir(
60         getattr(sec, "upload_dir", None) or "output/uploads"
61     )
62     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
63     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
64     exts = [
65         e.lower().lstrip(".")
66         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
67     ]
68     return upload_dir, max_bytes, part_bytes, exts
69
70
71 def _abort_upload(upload_id: str) -> None:
72     with _uploads_lock:
73         st = _uploads.pop(upload_id, None)
74     if st:
75         try:
76             os.remove(st["tmp_path"])
77         except OSError:
78             pass
79
80
81 @router.post("/api/upload/init")
82 def upload_init(req: UploadInitRequest):
83     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
84     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
85     try:
86         name = safe_output_filename(req.filename)
87     except ValueError as e:
88         raise HTTPException(status_code=400, detail=str(e)) from e
89     ext = os.path.splitext(name)[1].lower().lstrip(".")
90     if ext not in exts:
91         raise HTTPException(
92             status_code=400,
93             detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
94         )
95     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
96         raise HTTPException(
97             status_code=413,
98             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
99         )
100     os.makedirs(upload_dir, exist_ok=True)
101     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
102     # effort ? an
103     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
104     # (never blocks a
105     # legit upload); the per-part total cap in upload_part is the hard backstop either
106     # way.
107     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
108     upload_id = uuid.uuid4().hex
109     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
110     open(tmp_path, "wb").close()
111     with _uploads_lock:
112         _uploads[upload_id] = {
113             "filename": name,
114             "tmp_path": tmp_path,
115             "parts": 0,
116             "bytes": 0,
117         }
118     return {
119         "upload_id": upload_id,
120         "max_part_mb": part_bytes // (1024 * 1024),

```

```

118         "next_part": f"/api/upload/{upload_id}/part/0",
119     }
120
121
122 @router.put("/api/upload/{upload_id}/part/{index}")
123 async def upload_part(upload_id: str, index: int, request: Request):
124     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
125     _, max_bytes, part_bytes, _ = _upload_cfg()
126     with _uploads_lock:
127         st = _uploads.get(upload_id)
128         if st is None:
129             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
130         if index != st["parts"]:
131             raise HTTPException(
132                 status_code=409,
133                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
134             )
135
136         received = 0
137         overflow = None
138         with open(st["tmp_path"], "ab") as fh:
139             async for chunk in request.stream():
140                 received += len(chunk)
141                 if received > part_bytes:
142                     overflow = (
143                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
144                     )
145                     break
146                 if st["bytes"] + received > max_bytes:
147                     overflow = (
148                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
149                     )
150                     break
151                 fh.write(chunk)
152         if overflow:
153             _abort_upload(upload_id)
154             raise HTTPException(
155                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
156             )
157
158         with _uploads_lock:
159             st["parts"] += 1
160             st["bytes"] += received
161         return {
162             "upload_id": upload_id,
163             "received_parts": st["parts"],
164             "received_bytes": st["bytes"],
165         }
166
167
168 @router.post("/api/upload/{upload_id}/complete")
169 def upload_complete(upload_id: str, req: UploadCompleteRequest):
170     """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
171     upload_dir, _, _, _ = _upload_cfg()
172     with _uploads_lock:
173         st = _uploads.get(upload_id)
174         if st is None:
175             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
176         if st["parts"] == 0 or st["parts"] != req.total_parts:
177             raise HTTPException(
178                 status_code=409,
179                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
180             )

```

```

181     final_path = os.path.join(upload_dir, st["filename"])
182     if os.path.exists(
183         final_path
184     ): # never overwrite an earlier upload with the same name
185         stem, ext = os.path.splitext(st["filename"])
186         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
187     os.replace(st["tmp_path"], final_path)
188     with _uploads_lock:
189         _uploads.pop(upload_id, None)
190     video_id = compute_video_id(final_path)
191     return {
192         "path": os.path.abspath(final_path),
193         "video_id": video_id,
194         "size_bytes": os.path.getsize(final_path),
195         "next": "POST /api/process with this path",
196     }
197
198
199 @router.delete("/api/upload/{upload_id}")
200 def upload_abort(upload_id: str):
201     """Abort an in-flight upload and delete its partial file."""
202     with _uploads_lock:
203         known = upload_id in _uploads
204     if not known:
205         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
206     _abort_upload(upload_id)
207     return {"status": "aborted", "upload_id": upload_id}
208

```

9. user (2026-07-02T04:54:30.439Z)

```

1     """Resolved application-data HOME for a packaged, batteries-included install.
2
3     PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4     the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5     ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
6     from
7     the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
8     silently
9     "loses" their index. This module introduces ONE resolver every state path roots on.
10
11     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
12     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
13     sets it).
14     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
15     behaviour.
16
17     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
18     ``~/local/share``)
19     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
20     launcher (P2)
21     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
22     importing
23     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
24     writes into a
25     real user profile during a test run).
26
27     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
28     before
29     the heavier pydantic-backed ``backend.config`` package ? keeping the early
30     ``load_dotenv`` path light.
31
32     """
33
34     from __future__ import annotations

```

```

25     import os
26     import platform
27     from pathlib import Path
28
29     #: Environment variable the packaged launcher sets to pin the app-data home.
30     APP_HOME_ENV = "RALLY_APP_HOME"
31
32     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33     APP_DIR_NAME = "Khelsutra"
34
35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """``<app-home>/env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if (
76             str(home) == "."
77         ): # env unset (or pinned to CWD) ? historical load_dotenv() search
78             return None
79         candidate = home / ".env"
80         return candidate if candidate.exists() else None

```

```

81
82
83     def resolve_output_dir(output_dir: str) -> str:
84         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
85
86         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
87         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
88         normalisation of ``/x`` / separators / trailing slash). With it SET, a relative dir
like
89         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
90         app's state; an absolute path ? or a Windows drive-anchored one (``C:\x`` /
``C:x``) ? passes
91         through untouched.
92         """
93         home = app_home()
94         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
95             return output_dir or "output"
96         if not output_dir:
97             return str(home / "output")
98         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
99             return output_dir
100         return str(home / output_dir)
101
102
103     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
105
106         * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
107         * macOS ? ``~/Library/Application Support/<app_name>``
108         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
109         """
110
111         system = platform.system()
112         if system == "Windows":
113             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114         elif system == "Darwin":
115             base = str(Path.home() / "Library" / "Application Support")
116         else:
117             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118         return Path(base) / app_name
119

```

10. user (2026-07-02T04:55:23.337Z)

C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:5:under security.upload_dir, then submitted to /api/process by the returned path.

C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:6:A RELATIVE upload_dir roots under the app home (RALLY_APP_HOME) via the same

C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:10:aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir

C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:54:
P0a: root a RELATIVE upload_dir under the app home (RALLY_APP_HOME), exactly like

C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:58:

```

# an absolute upload_dir always passes through untouched.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:59:
upload_dir = resolve_output_dir(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:60:
getattr(sec, "upload_dir", None) or "output/uploads"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:68:
return upload_dir, max_bytes, part_bytes, exts
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\api\uploads.py:84:
upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:100:    os.makedirs(upload_dir, exist_ok=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:104:    require_free_bytes(upload_dir, req.total_size_bytes,
stage="upload")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:106:    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:171:    upload_dir, _, _, _ = _upload_cfg()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:181:    final_path = os.path.join(upload_dir, st["filename"])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\api\uploads.py:186:    final_path = os.path.join(upload_dir,
f"{stem}-{upload_id[:8]}{ext}")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tools\cleanup_caches.py:5:(`security.upload_dir`, default `output/uploads`) ? classifies
each entry as
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tools\cleanup_caches.py:30:automatically when `wsl` is unavailable). ``--uploads-dir``
must match `security.upload_dir`
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tools\cleanup_caches.py:56:    # The upload landing zone (security.upload_dir). Uploaded
source videos and the
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tools\cleanup_caches.py:274:    Flat by design today (uploads.py writes
`<upload_dir>/<filename>` plus
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tools\cleanup_caches.py:403:    help="upload landing zone to sweep (default <output-
dir>/uploads; match security.upload_dir)",
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\config\models.py:337:    # ? In hosted mode, set allowed_video_root to contain
upload_dir or /api/process
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\config\models.py:339:    upload_dir: str = "output/uploads"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\utils\freespace.py:4:`upload_dir`), a **fetch-to-scratch** of a bucket/Drive
source (a full local copy under the temp
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

```

```

indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\backend\utils\freespace.py:32: ancestor (an ``upload_dir``/scratch target may not exist
yet at guard time)."""
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\docs\VIDEO_LOCALITY_MODEL.md:13:assembles the full file under `security.upload_dir` (default
`output/uploads`) on
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\docs\CODE_MAP.md:115:[Omitted long matching line]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\docs\COMPUTE_DECOUPLED_SERVING\README.md:170:[Omitted long matching line]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:33: cfg.security.upload_dir = str(tmp_path / "up")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:127: f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:208: f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:234: f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:238:# --- P0a: upload_dir roots under the app home
(RALLY_APP_HOME) -----
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:246: monkeypatch.setattr(m, "global_config", AppConfig())
# upload_dir at its default
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:247: upload_dir, _, _, _ = uploads_api._upload_cfg()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:248: assert upload_dir == str(tmp_path / "output" /
"uploads")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:251:def
test_upload_cfg_absolute_upload_dir_passes_through(monkeypatch, tmp_path):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:255: cfg.security.upload_dir = abs_dir
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:257: upload_dir, _, _, _ = uploads_api._upload_cfg()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:258: assert upload_dir == abs_dir
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:265: upload_dir, _, _, _ = uploads_api._upload_cfg()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:266: assert upload_dir == "output/uploads"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:270: # End-to-end packaged shape: RALLY_APP_HOME set,

```

```
upload_dir left at its relative
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-
dir\tests\test_upload_download.py:279:   cfg = AppConfig() # security.upload_dir stays
'output/uploads'
```

11. user (2026-07-02T04:55:24.557Z)

```
93: def create_app(config: AppConfig, db: Database) -> FastAPI:
94-     """Factory: inject shared state via ``app.state`` so route handlers access it through
95-     ``request.app.state`` ? eliminating the Optional module-level globals that forced
96-     ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
97-
98-     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
99-     compatibility (helpers and tests still reference them). Route handlers use
100-     ``request.app.state`` exclusively."""
101-     global global_config, db_instance
102-     global_config = config
103-     db_instance = db
104-     app.state.config = config
105-     app.state.db = db
106-     return app
107-
108-
109- def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
110-     """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
111-     separated),
112-     default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
113-     ``backend.main``
114-     does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
115-     origin."""
116-     src = os.environ if env is None else env
117-     raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
118-     origins = [o.strip() for o in raw.split(",") if o.strip()]
119-     return origins or ["*"]
120-
121-
122- # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by browsers
123- # per the
124- # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
125- # credentials stay
126- # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
127- # deploy can
128- # lock it to its origin; the middleware is fixed at startup.
129- app.add_middleware(
```

12. user (2026-07-02T04:55:36.379Z)

```
1     """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3     Contract under test:
4     - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5     - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
6     (413)
7     - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
8     - DELETE /api/upload/{id} ? abort removes partial state
9     - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
10
11
12     import os
13     from pathlib import Path
14     from unittest.mock import patch
15
16     import pytest
```

```

17     pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
18     from fastapi.testclient import TestClient
19
20     import backend.main as m
21     from backend.api import uploads as uploads_api
22     from backend.config.models import AppConfig
23     from backend.infrastructure.database import Database
24     from backend.utils import app_home as ah
25     from backend.utils.hashing import compute_video_id
26
27
28     @pytest.fixture
29     def env(tmp_path, monkeypatch):
30         db = Database(str(tmp_path / "t.db"))
31         cfg = AppConfig()
32         cfg.output.output_dir = str(tmp_path / "out")
33         cfg.security.upload_dir = str(tmp_path / "up")
34         os.makedirs(cfg.output.output_dir, exist_ok=True)
35         monkeypatch.setattr(
36             uploads_api, "_uploads", {}
37         ) # isolate upload state per test (registry moved to backend.api.uploads)
38         app = m.create_app(cfg, db)
39         client = TestClient(app)
40         return client, db, tmp_path, cfg
41
42
43     def _init(client, filename="match.mp4", total=None):
44         body = {"filename": filename}
45         if total is not None:
46             body["total_size_bytes"] = total
47         return client.post("/api/upload/init", json=body)
48
49
50     # --- init -----
51
52
53     def test_init_rejects_bad_extension(env):
54         client = env[0]
55         r = _init(client, "malware.exe")
56         assert r.status_code == 400
57         assert "not allowed" in r.json()["detail"]
58
59
60     def test_init_rejects_traversal_filename(env):
61         client = env[0]
62         assert _init(client, "../evil.mp4").status_code == 400
63         assert _init(client, "a/b.mp4").status_code == 400
64
65
66     def test_init_rejects_oversize_declared_total(env):
67         client, db, tmp_path, cfg = env
68         cfg.security.max_upload_mb = 1
69         r = _init(client, total=2 * 1024 * 1024)
70         assert r.status_code == 413
71
72
73     def test_init_507_when_disk_too_full_for_declared_total(env):
74         # P2 (D11): a declared upload that won't fit fails fast with 507 (not a mid-upload
75         crash).
76         import types
77
78         from backend.utils import freespace
79
80         client = env[0]
81         with patch.object(

```

13. user (2026-07-02T04:55:58.820Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:731:
SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root`` is set ? the
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:747: if
getattr(sec, "allowed_video_root", None):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:752:
"(security.allowed_video_root is set); upload to local instead."
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:1373:
getattr(request.app.state.config, "security", None), "allowed_video_root", None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:1388: #
input: its null-byte guard always applies, and a configured allowed_video_root (hosted mode)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\main.py:1797:
auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\models.py:330: allowed_video_root: Optional[str] = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\models.py:337: # ? In hosted mode, set allowed_video_root to contain
upload_dir or /api/process
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\startup.py:22:rest of the posture at **boot**: the ``allowed_video_root``
path-jail is set (else an authenticated
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\startup.py:107: # a tenant can point the engine at an ARBITRARY local file
(allowed_video_root unset = "any local
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\startup.py:109: root = getattr(sec, "allowed_video_root", None)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\startup.py:115: "security.edge_auth_enabled is ON (engine
exposed) but security.allowed_video_root is "
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\config\startup.py:117: "Set allowed_video_root to a jail directory
before exposing the engine.",
```

14. user (2026-07-02T04:56:00.447Z)

```
        help="protect entries whose name contains this id/substring",
    )
    ap.add_argument(
        "--older-than",
        type=float,
        default=0.0,
        help="only purge regenerable caches older than N days",
    )
    ap.add_argument(
        "--include-wasbcache",
        action="store_true",
        help="also purge detector resume caches",
    )
    ap.add_argument(
        "--stage-dir", default=~/.clips", help="WSL stage dir (default ~/.clips)"
    )
)
```

```

ap.add_argument("--distro", default="Ubuntu", help="WSL distro (default Ubuntu)")
ap.add_argument(
    "--output-dir", default="output", help="Windows output dir (default output)"
)
ap.add_argument(
    "--uploads-dir",
    default=None,
    help="upload landing zone to sweep (default <output-dir>/uploads; match
security.upload_dir)",
)
ap.add_argument(
    "--uploads-retention-days",
    type=float,
    default=7.0,
    help="sweep uploaded source videos older than N days (default 7; 0=off). The T11 privacy
window.",
)
ap.add_argument(
    "--summary",
    action="store_true",
    help="print a single-line nudge (only if reclaimable >= --threshold-gb); for hooks",
)
ap.add_argument(
    "--threshold-gb",
    type=float,
    default=5.0,
    help="--summary stays silent below this many reclaimable GB (default 5)",
)

```

15. user (2026-07-02T04:56:07.711Z)

```

1360     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1361     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1362     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1363     """"
1364     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
1365     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
1366     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
1367     try:
1368         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1369     except edge_auth.EdgeAuthError as e:
1370         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1371
1372     allowed_root = getattr(
1373         getattr(request.app.state.config, "security", None), "allowed_video_root", None
1374     )
1375     try:
1376         validate_input_path(req.video_path, allowed_root=allowed_root)
1377     # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
1378     # scheme, which we map to a clean 400 (an unknown scheme must never 500).
1379     input_ref = storage.resolve_input_ref(
1380         req.video_path, getattr(request.app.state.config, "storage", None)
1381     )
1382     except ValueError as e:
1383         raise HTTPException(status_code=400, detail=str(e)) from e
1384     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
1385     # (input_ref.key ? e.g. local:/// stripped to the bare path), not the raw URI
string. A
1386     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
1387     # fetches it (a fetch miss fails the job loudly). validate_input_path above still

```

```

runs for every
1388     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
1389     # rejects a non-local URI as out-of-root.
1390     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1391         raise HTTPException(
1392             status_code=404, detail=f"Video file not found at '{req.video_path}'"
1393         )
1394     # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
1395     # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
1396     # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
1397     # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
1398     if input_ref.backend != "local":
1399         need = _remote_input_size_bytes(
1400             input_ref, getattr(request.app.state.config, "storage", None)
1401         )
1402         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
1403     if req.segmenter and not segmenter_registry.get(req.segmenter):
1404         available = list(segmenter_registry.list_available().keys())

```

16. user (2026-07-02T04:56:23.679Z)

```

a8e191e api: root a relative security.upload_dir under the app home (P0a parity with output_dir)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
backend/api/uploads.py          | 13 ++++++
backend/config/models.py        | 9 +++++
docs/CODE_MAP.md                | 2 +-
tests/test_upload_download.py   | 61 +++++
4 files changed, 80 insertions(+), 5 deletions(-)

```

17. user (2026-07-02T04:56:29.956Z)

```

..... [100%]
46 passed in 3.55s

```

18. user (2026-07-02T04:56:40.471Z)

```

All checks passed!
3 files already formatted

```

19. user (2026-07-02T04:56:45.027Z)

```

backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 2 source files

```

20. assistant (2026-07-02T04:56:50.755Z)

Lint and types clean. Now the full suite per the pre-LGTM gate.

21. user (2026-07-02T04:56:55.075Z)

```
run_all_tests.py
```

22. user (2026-07-02T04:57:03.843Z)

Command running in background with ID: bdhieag6x. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\tasks\bdhieag6x.output. You will be notified when it completes. To check interim output, use Read on that file path.

23. assistant (2026-07-02T04:57:43.476Z)

While the suite runs, let me ground the `allowed\_video\_root` interaction claim.

24. user (2026-07-02T04:57:45.347Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:58:def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:59-
"""Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:60-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:61-
With no allowed_root (default, single-user local mode) only the null-byte check applies,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:62-
so the local 'point it at any of my videos' workflow is preserved. Setting an
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:63-
allowed_root (a future hosted/tenant deployment) enforces containment.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:64-
"""
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:65-
if "\x00" in path:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:66-
raise ValueError("video_path must not contain a null byte")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:67-
if allowed_root:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-upload-dir\backend\utils\paths.py:68-
resolve_under_root(path, allowed_root)
```

25. user (2026-07-02T04:57:55.492Z)

```
725
726
727     @app.post("/api/assets", status_code=201)
728     def create_asset(req: AssetRequest, request: Request):
729         """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).
730
731         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
732         hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
733         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
734         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
735         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
736         stays as the working copy; a ``local`` medium keeps the upload in place (it's
```

```

already on this box);
737         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
738         MD5 (`compute_video_id`, D3) so it joins the pipeline by content hash, and appears
in
739         ``GET /api/videos``.
740     try:
741         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
742     except edge_auth.EdgeAuthError as e:
743         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
744
745     cfg = request.app.state.config
746     sec = getattr(cfg, "security", None)
747     if getattr(sec, "allowed_video_root", None):
748         raise HTTPException(
749             status_code=403,
750             detail=(
751                 "storing into a custom medium is disabled on this hosted deployment "
752                 "(security.allowed_video_root is set); upload to local instead."
753             ),
754         )
755     if bool(req.uploaded_path) == bool(req.source_uri):
756         raise HTTPException(
757             status_code=400,
758             detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
759         )
760     if req.uploaded_path and not req.medium_uri:
761         raise HTTPException(
762             status_code=400, detail="'medium_uri' is required when uploading a file."
763         )
764
765     storage_settings = _storage_settings(cfg)
766
767     # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
768     if req.uploaded_path:
769         try:
770             validate_input_path(req.uploaded_path, allowed_root=None)
771             src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
772         except ValueError as e:
773             raise HTTPException(status_code=400, detail=str(e)) from e
774         if src_ref.backend != "local" or not os.path.isfile(src_ref.key):

```

26. user (2026-07-02T04:58:05.214Z)

```

#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```

python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage

```

Anything after the script name is passed straight through to pytest.

Notes

- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport` package isn't installed (they use `pytest.importorskip`). To exercise them too:

```

pip install -e ../sports-obsreport --no-build-isolation

```
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).

```
"""
```

```
from __future__ import annotations
```

```
import importlib.util
```

```
import subprocess
```

```
import sys
```

```
def _installed(module: str) -> bool:
```

```
    try:
```

```
        return importlib.util.find_spec(module) is not None
```

```
    except (ImportError, ValueError):
```

```
        return False
```

```
def main() -> int:
```

```
    args = list(sys.argv[1:])
```

```
    # Default to quiet unless the caller chose a verbosity.
```

```
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
```

27. user (2026-07-02T04:58:29.541Z)

```
---
```

28. assistant (2026-07-02T04:58:34.890Z)

You've hit your session limit · resets 1:10pm (Asia/Calcutta)